Zgodnie z wytycznymi dotyczącymi analizy własnego doświadczenia w realizacji projektów, poniżej przedstawiamy przebieg oraz wnioski z prac nad naszym systemem. Realizowaliśmy projekt grupowy, którego celem było stworzenie w pełni funkcjonalnej aplikacji webowej do zarządzania projektami. Zespół składał się z kilku osób, a prace podzieliliśmy według standardowego modelu na rozwój frontendy, logiki serwera oraz bazy danych, gdzie backend oparto na frameworku Django.

Początkowo prace przebiegały bardzo dynamicznie. Szybko przygotowaliśmy makiety wizualne i rozpoczęliśmy programowanie poszczególnych modułów. Zależało nam na czasie, dlatego każda podgrupa pracowała w dużej izolacji, skupiając się na własnym wycinku aplikacji. Framework Django pozwalał na błyskawiczne generowanie i wystawianie API, co dało nam złudne poczucie bezpieczeństwa.

Problemy zaczęły się jednak już w połowie terminu przewidzianego na realizację. Zespół napotkał barierę komunikacyjną przy próbie łączenia obu warstw, a wykonana wcześniej dekompozycja zadań analitycznych okazała się niepoprawna. Domyślna struktura i format danych z API generowanego przez Django znacząco mijały się z oczekiwaniami frontendu. Co więcej, przez pośpiech zignorowaliśmy dobre praktyki korzystania z systemu kontroli wersji (Git). Członkowie zespołu często zgrywali swoje zmiany bezpośrednio do głównej gałęzi kodu, co z powodu braku mechanizmów weryfikacji doprowadziło do ogromnych konfliktów w plikach.

Tworząc system zarządzania, w początkowym procesie analizy nie uwzględniliśmy odpowiednio czynników ludzkich oraz ryzyka błędów koordynacyjnych. Ostatnie tygodnie projektu, począwszy od połowy terminu, zamieniły się w gorączkowe dostosowywanie kodu po stronie klienckiej do wymogów API w Django oraz żmudne, ręczne naprawianie struktury repozytorium. Projekt ostatecznie udało się wdrożyć, jednak kosztowało to zespół dużo dodatkowej pracy.

Wnioski z realizacji

Realizacja witryny pozwoliła wyciągnąć następujące wnioski, istotne przy kolejnych przedsięwzięciach:

- **Ostrożność przy zadaniach integracyjnych:** Oszacowanie nakładu pracy na zadania zawierające pojęcie „integracji” (nawet pomiędzy własnymi komponentami frontend/backend) zawsze wiąże się z koniecznością dokładniejszej oceny i analizy struktury danych.
- **Mechaniczna weryfikacja i dokumentacja:** Przed napisaniem kodu należy precyzyjnie ustalić kontakty API. Zespoły muszą dysponować wspólną dokumentacją, a weryfikacja powstających zmian powinna być szybka i mechaniczna (np. poprzez zautomatyzowane testy zgodności).

- **Wewnętrzna samokontrola:** Role w projekcie powinny być zdefiniowane tak, aby pozwalały na wewnętrzną samokontrolę zespołu. W przypadku systemu Git konieczne jest stosowanie twardych procedur (np. Pull Requests i wzajemne sprawdzanie kodu przed jego połączeniem), aby zapobiec konfliktom.
- **Szacowanie ryzyka i czynnik ludzki:** Należy dokonywać holistycznej analizy projektu już na starcie i zawsze uwzględniać w niej czynniki ludzkie oraz potencjalne ryzyka komunikacyjne.